

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



PROGRAMMING FUNDAMENTALS - CO1027

ASSIGNMENT 2

ONE PIECE

Arc Water 7 - Enies Lobby (part 2)

Version 1.0

ASSIGNMENT SPECIFICATION

Version 1.0

1 Learning outcomes

After completing this assignment, students review and make good use of:

- File Input/Output
- Pointer and dynamic memory allocation
- Object-oriented programming
- Singly linked list

2 Introduction

In the previous part, we followed the Straw Hat Pirates on their journey at Water 7, where internal conflicts gradually surfaced and critical decisions were made. The climax of these events occurred when Nico Robin was captured by the World Government and taken to Enies Lobby—one of the three most powerful judicial institutions of the Government.

At Enies Lobby, the Straw Hat Pirates are forced into a direct confrontation with the elite CP9 agents in order to rescue their crewmate. This battle is not merely a clash between individuals, but also a struggle against time, the battlefield environment, and various tactical factors. Structures such as the Main Gate, the Courthouse, the Tower of Justice, and the Bridge of Hesitation play crucial roles, directly influencing the progress of the rescue mission.

In this assignment, students are required to simulate the battle at Enies Lobby by designing classes and implementing interaction mechanisms among characters, buildings, and the battle state. Characters will act in turns, performing attacks or using special skills, while buildings continuously affect the overall progression of the battle.

3 Classes in the Program

This assignment applies Object-Oriented Programming to model the battle in which the Straw Hat Pirates attempt to rescue Nico Robin at Enies Lobby. The entities in this assignment are represented as classes and are described as follows.

Note:

- Students are required to carefully read the problem statement to determine the inheritance relationships (derived classes), as these will not be explicitly specified.
- Students are allowed to define additional attributes, methods, or classes as needed to support the implementation of the required classes in this assignment.
- Within the scope of this assignment, whenever computed values need to be rounded to integers, students must round up.
- Within the scope of this assignment, the values of `hp`, `energy`, `morale`, `alarmLevel`, `rescueProgress`, `escapeProgress`, and the `hp` of buildings must always be clamped to their valid ranges after each update. Specifically, a character's `hp` must lie within the range from 0 to `maxHp`, a building's `hp` must lie within the range from 0 to `maxHP`, and `energy`, `morale`, `alarmLevel`, `rescueProgress`, and `escapeProgress` must lie within the range from 0 to 100. In addition, `busterCallTimer` must not be less than 0.

3.1 Character

Each character participating in the battle at Enies Lobby is represented by an abstract base class `Character`. This class serves as the parent class for all characters in the program, including members of the Straw Hat Pirates and CP9 agents. :contentReference[oaicite:0]index=0

Requirement: Implement the abstract class `Character` with the following specifications:

1. `protected` attributes:

- `name`: type `string`, the name of the character.
- `hp`: type `int`, the current health points of the character.
- `maxHp`: type `int`, the maximum health points of the character.
- `atk`: type `int`, the base attack value.
- `def`: type `int`, the base defense value.
- `speed`: type `int`, the action speed of the character.
- `energy`: type `int`, the current energy used to perform skills.
- `alive`: type `bool`, the combat status of the character.

2. A public constructor with parameters `name`, `hp`, `atk`, `def`, `speed`, `energy` corresponding to the attributes with the same names. The constructor assigns the given parameters to the corresponding attributes. The value of `maxHp` is initialized to `hp`. If `hp` is greater than 0, then `alive` is set to `true`; otherwise, it is set to `false`.

```
1 Character(string name, int hp, int atk, int def, int speed, int energy);
```

3. A public virtual destructor.

```
1 virtual ~Character();
```

4. Pure virtual method **attack** used to perform a normal attack on another character. The method takes a pointer **target** representing the attacked character and a reference **context** representing the current battle state. The return value is the actual damage dealt.

```
1 virtual int attack(Character* target, BattleContext& context) = 0;
```

5. Pure virtual method **specialSkill** used to perform the character's special ability. Each derived class will implement this method differently. The method takes a pointer **target** representing the affected entity and a reference **context** representing the current battle state. The return value is the amount of damage dealt.

```
1 virtual int specialSkill(Character* target, BattleContext& context) = 0;
```

6. Virtual method **attack** with parameter **Building*** used to perform an attack on a building.

This method has a default implementation that deals no damage and returns 0. Derived classes representing characters capable of affecting buildings are required to override this method.

```
1 virtual int attack(Building* target, BattleContext& context);
```

7. Virtual method **specialSkill** with parameter **Building*** used to perform a special skill on a building.

This method has a default implementation that deals no damage and returns 0. Derived classes representing characters capable of affecting buildings are required to override this method.

```
1 virtual int specialSkill(Building* target, BattleContext& context);
```

8. Virtual method **endTurn** used to process end-of-turn effects for the character. These effects may include restoring energy, updating states, or modifying certain values in **BattleContext**.

```
1 virtual void endTurn(BattleContext& context);
```

9. Method **receiveDamage** used to update the character's health when taking damage. The actual damage received is calculated as the difference between the input **damage** and the character's defense value. If this difference is less than 0, the character takes no damage. If after taking damage, **hp** is less than or equal to 0, then **hp** is set to 0 and **alive** is set to **false**.

```
1 void receiveDamage(int damage);
```

10. Method `isAlive` returns whether the character is still able to fight.

```
1 bool isAlive() const;
```

11. Method `getName` returns the name of the character.

```
1 string getName() const;
```

12. Method `getHP` returns the current health points of the character.

```
1 int getHP() const;
```

13. Method `getEnergy` returns the current energy of the character.

```
1 int getEnergy() const;
```

14. Virtual method `isStrawHat` returns `true` if the character belongs to the Straw Hat Pirates, and `false` otherwise. The class `StrawHat` must override this method.

```
1 virtual bool isStrawHat() const;
```

15. Virtual method `isCP9` returns `true` if the character belongs to CP9, and `false` otherwise. The class `CP9Agent` must override this method.

```
1 virtual bool isCP9() const;
```

16. Pure virtual method `str` returns a string representation of the character.

```
1 virtual string str() const = 0;
```

3.2 Straw Hat Pirates

The Straw Hat Pirates are the primary force in the battle at Enies Lobby, with the objective of rescuing Nico Robin. All members of the crew inherit from the `StrawHat` class, which is a derived class of `Character`. :contentReference[oaicite:0]index=0

Requirement: Implement the class `StrawHat` with the following specifications:

1. `protected` attributes:

- `bounty`: type `long long`, the character's bounty.

2. A `public` constructor that takes parameters identical to those of `Character`, with an additional parameter `bounty` used to initialize the corresponding attribute.

```
1 StrawHat(string name, int hp, int atk, int def,  
2         int speed, int energy, long long bounty);
```

3. Method `isStrawHat`:

```
1 virtual bool isStrawHat() const;
```

Returns `true` if the character belongs to the Straw Hat Pirates, and `false` otherwise. This method is used to determine the faction of a character during battle processing.

4. Method `str`:

```
1 virtual string str() const;
```

Returns a string representation of a Straw Hat character. The returned string must follow the format:

```
1 StrawHat[name=<name>, hp=<hp>, atk=<atk>, def=<def>, speed=<speed>,  
    energy=<energy>, bounty=<bounty>]
```

All values must be printed consecutively, separated by commas, with no extra whitespace beyond the specified format.

Example:

```
1 StrawHat[name=Luffy, hp=120, atk=35, def=20, speed=25, energy=50, bounty  
    =150000000]
```

5. Methods inherited from `Character` will be overridden in specific derived classes.

6. When a crew member defeats a CP9 agent, if the character description does not specify any effect on `morale`, then the default behavior is that `morale` in `BattleContext` increases by 5.

3.2.1 Monkey D. Luffy

Luffy is the captain of the Straw Hat Pirates, possessing immense combat strength and becoming increasingly dangerous when placed in disadvantageous situations.

Requirement: Implement class `Luffy` derived from `StrawHat`.

- Method `attack`:

The damage dealt depends on Luffy's current health percentage. If current health is greater than 50% of maximum health, the damage equals the attack value. If current health is greater than 30% and less than or equal to 50%, the damage increases by 15%. If current health is less than or equal to 30%, the damage increases by 30%.

If the attack defeats the target, `morale` in `BattleContext` increases by 5.

- Method `specialSkill`:

Luffy uses Gear Second at a cost of 20 energy. This skill deals damage equal to 200% of the attack value, while increasing Luffy's speed and damage by 15. Using this skill

increases `alarmLevel` by 10 and reduces Luffy's health by an amount equal to 8% of maximum health.

This skill can only be used if Luffy has at least 20 energy and his health is not less than 15% of maximum health.

- Method `endTurn`:

If current health is less than or equal to 30% of maximum health, `morale` increases by 3.

If Luffy defeats an opponent during the turn, he recovers 5 energy.

3.2.2 Roronoa Zoro

Zoro is the swordsman of the Straw Hat Pirates, capable of dealing high damage and efficiently finishing off targets.

Requirement: Implement class `Zoro` derived from `StrawHat`.

- Method `attack`:

Zoro's damage equals his attack value plus an additional amount equal to 20% of his defense. If the target's health is less than 40% of its maximum health, the damage increases by 15%.

- Method `specialSkill`:

Zoro uses a skill with a cost of 15 energy, dealing damage equal to 220% of his attack value. If the target's health is less than 50% of its maximum health, the skill's damage increases by 50%. If this skill defeats the target, Zoro recovers 8 energy and `morale` increases by 4.

- Method `endTurn`:

If Zoro defeats a target, `morale` increases by 6 and Zoro's attack increases by 5%.

3.2.3 Vinsmoke Sanji

Sanji is a chef who fights using powerful kicking techniques, capable of dealing sustained damage and weakening enemies.

Requirement: Implement class `Sanji` derived from `StrawHat`.

- Method `attack`:

Damage equals the attack value plus an additional amount equal to 50% of current speed. If the target's defense is lower than Sanji's, the damage increases by 10%. This rule does not apply to buildings.

- Method `specialSkill`:

Sanji uses a skill with a cost of 18 energy, dealing damage equal to 210% of the attack value. After being hit, the target's defense decreases by 8 (ignore if target is a building). If the target is Jabra, the defense decreases by 12 instead of 8.

- Method `endTurn`:

If the opponent is defeated, `morale` increases by 8 and Sanji's attack increases by 10%.

3.2.4 Nami

Nami is the navigator of the crew, capable of controlling the battlefield and applying debuffs.

Requirement: Implement class `Nami` derived from `StrawHat`.

- Method `attack`:

Damage equals the attack value while ignoring 30% of the target's defense. If the target is a building, the damage is equal to 50% of the attack value.

- Method `specialSkill`:

Nami uses a skill with a cost of 20 energy, dealing damage equal to her attack value plus 40. Additionally, it reduces the target's speed by 10. If the target is a building, the damage increases by 50%. After use, `busterCallTimer` increases by 1 and `alarmLevel` decreases by 5.

- Method `endTurn`:

If the opponent is defeated, Nami recovers 6 energy.

3.2.5 Tony Tony Chopper

Chopper is the crew's doctor, capable of both healing allies and engaging in combat.

Requirement: Implement class `Chopper` derived from `StrawHat`.

- Method `attack`:

Damage equals Chopper's attack value.

- Method `specialSkill`:

Chopper uses a healing skill with a cost of 15 energy. This skill can only be used on the Straw Hat member with the lowest health. The skill restores an amount equal to 35 plus 50% of Chopper's attack value.

If the target is Luffy, `morale` in `BattleContext` increases by 5.

- Method **endTurn**:
Chopper has no special end-of-turn effects.

3.2.6 Usopp (Sogeking)

Due to a previous conflict with Luffy, Usopp disguises himself as Sogeking during the battle. Sogeking is a sniper capable of applying effects and providing tactical support.

Requirement: Implement class **Usopp** derived from **StrawHat**.

- Method **attack**:
Damage equals the attack value. If the target's speed is less than 50, the damage increases by 20%. If the target is a building, the damage is equal to 50% of the attack value.
- Method **specialSkill**:
Sogeking uses a skill with a cost of 16 energy, dealing damage equal to 80% of the attack value. Additionally, it reduces the target's speed by 12 (ignore if target is a building). After use, **escapeProgress** increases by 8.
- Method **endTurn**:
After Sogeking attacks, the team's **morale** increases by 10.

3.2.7 Franky (Cutty Flam)

Franky is the leader of a group of outlaws on Water Seven and later joins Luffy's crew during the mission to rescue Robin. Franky is capable of dealing high damage and affecting a wide area.

Requirement: Implement class **Franky** derived from **StrawHat**.

- Method **attack**:
Damage equals the attack value plus an additional amount equal to 30% of defense. If the target belongs to CP9, the damage increases by 10%.
- Method **specialSkill**:
Franky has two skills:
Strong Right consumes 20 energy, dealing damage equal to 180% of the attack value and reducing the target's speed by 8 (does not apply to buildings). If the target is Lucci, the damage increases by 20%.
Coup de Vent consumes 30 energy, dealing damage equal to 120% of the attack value. If the target is a **Building**, that building is immediately destroyed.

- Method `endTurn`:

If current health is greater than 70% of maximum health, Franky's defense increases by 5.

If current health is less than 30% of maximum health, Franky's attack increases by 10%.

3.3 Cipher Pol Number 9 (CP9)

CP9 is an elite intelligence agency of the World Government, tasked with stopping the Straw Hat Pirates and protecting Enies Lobby. All CP9 members inherit from the `CP9Agent` class, which is a derived class of `Character`. :contentReference[oaicite:0]index=0

Requirement: Implement the class `CP9Agent` with the following specifications:

1. `protected` attributes:

- `doriki`: type `int`, representing the overall combat power of the agent.

2. A constructor that takes parameters identical to those of `Character`, with an additional parameter `doriki` used for initialization.

```
1 CP9Agent(string name, int hp, int atk, int def,
2          int speed, int energy, int doriki);
```

3. Method `isCP9`:

```
1 virtual bool isCP9() const;
```

Returns `true` if the character belongs to CP9, and `false` otherwise. This method is used to determine the faction of a character during battle processing.

4. Method `str`:

```
1 virtual string str() const;
```

Returns a string representation of a CP9 agent. The returned string must follow the format:

```
1 CP9[name=<name>, hp=<hp>, atk=<atk>, def=<def>, speed=<speed>, energy=<energy>, doriki=<doriki>]
```

All values must be printed consecutively, separated by commas, with no extra whitespace beyond the specified format. Example:

```
1 CP9[name=Lucci, hp=150, atk=40, def=25, speed=30, energy=60, doriki=4000]
```

5. When a CP9 agent defeats a Straw Hat member, if the agent's description does not specify any effect on `morale`, then by default `morale` in `BattleContext` decreases by 5.

3.3.1 Rob Lucci

Lucci is the strongest agent of CP9, possessing overwhelming power and the ability to eliminate targets swiftly.

Requirement: Implement class `Lucci` derived from `CP9Agent`.

- Method `attack`:

Damage equals the attack value plus an additional amount equal to one twentieth of `doriki`. If the target's health is less than 50% of its maximum health, the damage increases by 20%.

- Method `specialSkill`:

Lucci uses a skill with a cost of 25 energy, dealing damage equal to 280% of the attack value. This attack ignores 50% of the target's defense. If the skill defeats the target, `morale` decreases by an additional 10.

- Method `endTurn`:

If current health is less than 40% of maximum health, Lucci's attack increases by 5%.

3.3.2 Kaku

Kaku is a swordsman capable of executing consecutive attacks on a target.

Requirement: Implement class `Kaku` derived from `CP9Agent`.

- Method `attack`:

Damage equals the attack value.

- Method `specialSkill`:

Kaku uses a skill with a cost of 20 energy, performing three consecutive attacks on the same target.

The first hit deals damage equal to 120% of the attack value. The second hit deals damage equal to 100% of the attack value. The third hit deals damage equal to 80% of the attack value.

These attacks are executed sequentially. If the target is defeated before all three attacks are completed, the remaining attacks are not executed.

- Method `endTurn`:

Kaku has no special end-of-turn effects.

3.3.3 Jabra

Jabra is a defensive-oriented fighter with strong offensive capabilities.

Requirement: Implement class **Jabra** derived from **CP9Agent**.

- Method **attack**:
Damage equals the attack value.
- Method **specialSkill**:
Jabra uses a skill with a cost of 18 energy, dealing damage equal to 150% of the attack value.
If current health is less than 30% of maximum health, the skill's damage increases by 25%. If this attack defeats the target, **morale** decreases by an additional 5.
- Method **endTurn**:
Jabra has no special end-of-turn effects.

3.3.4 Blueno

Blueno is capable of evasive maneuvers and gaining advantages in combat.

Requirement: Implement class **Blueno** derived from **CP9Agent**.

- Method **attack**:
Damage equals the attack value.
- Method **specialSkill**:
Blueno uses a skill with a cost of 15 energy, dealing damage equal to 130% of the attack value.
If Blueno's current health is greater than 50% of maximum health, the skill's damage increases by 20. If current health is less than or equal to 50% of maximum health, the damage increases by 40.
- Method **endTurn**:
Blueno has no special end-of-turn effects.

3.3.5 Kalifa

Kalifa is capable of weakening opponents and reducing their fighting spirit.

Requirement: Implement class **Kalifa** derived from **CP9Agent**.

- Method **attack**:
Damage equals the attack value.
- Method **specialSkill**:
Kalifa uses a skill with a cost of 18 energy, dealing damage equal to 140% of the attack value. Additionally, it reduces **morale** by 8 and reduces the target's speed by 6. If the target is Nami, **morale** is reduced by 12 instead of 8.
- Method **endTurn**:
Kalifa has no special end-of-turn effects.

3.3.6 Kumadori

Kumadori is capable of dealing stable damage and becomes more effective under pressure.

Requirement: Implement class **Kumadori** derived from **CP9Agent**.

- Method **attack**:
Damage equals the attack value.
- Method **specialSkill**:
Kumadori uses a skill with a cost of 16 energy, dealing damage equal to 30 plus an additional amount equal to one tenth of **doriki**. If current health is less than 40% of maximum health, the damage increases by 25.
- Method **endTurn**:
Kumadori has no special end-of-turn effects.

3.3.7 Fukurou

Fukurou can track weakened targets and increase damage output.

Requirement: Implement class **Fukurou** derived from **CP9Agent**.

- Method **attack**:
Damage equals the attack value.
- Method **specialSkill**:
Fukurou uses a skill with a cost of 14 energy, dealing damage equal to 130% of the attack value. If the target is the character with the lowest health in the opposing team, the damage increases by 20. If this attack defeats the target, **morale** decreases by an additional 6.

- Method `endTurn`:
Fukurou has no special end-of-turn effects.

3.4 BattleContext

The `BattleContext` class stores the overall state of the entire battle at Enies Lobby. Both characters and buildings interact with this class to update progress and determine the outcome of the battle.

Requirement: Implement the class `BattleContext` with the following specifications:

1. public attributes:
 - `turnCount`: type `int`, the number of turns that have elapsed.
 - `morale`: type `int`, the morale level of the Straw Hat faction.
 - `alarmLevel`: type `int`, the alert level at Enies Lobby.
 - `rescueProgress`: type `int`, the progress of rescuing Robin.
 - `escapeProgress`: type `int`, the progress of escaping from Enies Lobby.
 - `busterCallTimer`: type `int`, the number of turns remaining before the Buster Call is triggered.
 - `mainGateDestroyed`: type `bool`, indicates whether the Main Gate has been destroyed. Default value is `false`.
 - `robinRescued`: type `bool`, indicates whether Robin has been rescued. Default value is `false`.
 - `bridgeOpened`: type `bool`, indicates whether the escape bridge has been opened. Default value is `false`.
 - `battleEnded`: type `bool`, indicates whether the battle has ended. Default value is `false`.
 - `resultCode`: type `string`, the final result of the battle.
2. A constructor that initializes all attributes.
3. Method `nextTurn` increments `turnCount` by 1 after each turn.

```
1 void nextTurn();
```

3.5 Building

Structures at Enies Lobby are modeled through the abstract class `Building`. Buildings do not participate directly in turn-based actions, but they influence the battle state through

BattleContext.

Requirement: Implement the abstract class `Building` with the following specifications:

1. protected attributes:

- `name`: type `string`, the name of the building.
- `hp`: type `int`, the current durability.
- `maxHP`: type `int`, the maximum durability.
- `destroyed`: type `bool`, indicates whether the building has been destroyed.

2. A constructor that takes parameters `name` and `hp`. The value of `maxHP` is initialized to the initial `hp`. If `hp` is greater than 0, `destroyed` is set to `false`; otherwise, it is set to `true`.

```
1 Building(string name, int hp);
```

3. A virtual destructor.

```
1 virtual ~Building();
```

4. Method `receiveDamage` is used to reduce the durability of the building. If after taking damage, `hp` is less than or equal to 0, `hp` is set to 0 and `destroyed` is set to `true`.

```
1 void receiveDamage(int damage);
```

5. Method `isDestroyed` returns the state of the building.

```
1 bool isDestroyed() const;
```

6. Pure virtual method `applyEffect` is called after each turn to apply the building's effects on `BattleContext`.

```
1 virtual void applyEffect(BattleContext& context) = 0;
```

7. Method `onDestroyed` is called when the building has just been destroyed to update the state in `BattleContext`.

```
1 virtual void onDestroyed(BattleContext& context);
```

8. Method `str`:

```
1 virtual string str() const;
```

Returns a string representation of the building. The returned string must follow the format:

```
1 Building[name=<name>, hp=<hp>, maxHP=<maxHP>, destroyed=<destroyed>]
```

All values must be printed consecutively, separated by commas, with no extra whitespace beyond the specified format. The value `destroyed` must be printed as `true` or `false`.

Example:

```
1 Building[name=MainGate, hp=0, maxHP=200, destroyed=true]
```

3.5.1 MainGate

The Main Gate is the primary entrance of Enies Lobby and serves as the first barrier for the Straw Hat Pirates.

Requirement: Implement class **MainGate** derived from **Building**.

- Method **applyEffect**:

If the Main Gate has not been destroyed, the rescue progress of Robin does not increase during this turn.

- Method **onDestroyed**:

When the Main Gate is destroyed, **mainGateDestroyed** is set to **true**, **rescueProgress** increases by 20, and **morale** increases by 5.

3.5.2 Courthouse

The Courthouse is the central area that increases the alert level.

Requirement: Implement class **Courthouse** derived from **Building**.

- Method **applyEffect**:

If the Courthouse has not been destroyed, **alarmLevel** increases by 5 after each turn.

- Method **onDestroyed**:

When destroyed, **alarmLevel** decreases by 20 and no longer increases in subsequent turns.

3.5.3 TowerOfJustice

The Tower of Justice is where Robin is being held.

Requirement: Implement class **TowerOfJustice** derived from **Building**.

- Method **applyEffect**:

If **mainGateDestroyed** is **true** and Robin has not yet been rescued, **rescueProgress** increases by 5 after each turn.

If **rescueProgress** is greater than or equal to 100, **robinRescued** is set to **true** and **morale** increases by 10.

3.5.4 BridgeOfHesitation

The Bridge of Hesitation is the escape route from Enies Lobby.

Requirement: Implement class `BridgeOfHesitation` derived from `Building`.

- Method `applyEffect`:

If `robinRescued` is `true`, `bridgeOpened` is set to `true` and `escapeProgress` increases by 5 each turn.

If `escapeProgress` is greater than or equal to 100, the battle ends with result `STRAW_HAT_WIN`.

3.5.5 BusterCallShip

The Buster Call Ship represents the time pressure of the battle.

Requirement: Implement class `BusterCallShip` derived from `Building`.

- Method `applyEffect`:

If the building has not been destroyed, `busterCallTimer` decreases by 1 after each turn.

If `busterCallTimer` is less than or equal to 0, the battle ends with result `BUSTER_CALL`.

- Method `onDestroyed`:

When the building is destroyed, `busterCallTimer` increases by 3.

3.6 EniesLobbyBattle

The `EniesLobbyBattle` class is responsible for managing the entire battle at Enies Lobby. This class handles reading initialization data, managing character lists, managing buildings, coordinating turn execution, updating the state in `BattleContext`, and determining the final result of the battle. `:contentReference[oaicite:0]index=0`

Requirement: Implement the class `EniesLobbyBattle` with the following specifications:

1. `private` attributes:

- `strawHats`: type `Character**`, a dynamic array storing Straw Hat characters. The number of Straw Hat members does not exceed 7.
- `strawHatCount`: type `int`, the current number of Straw Hat characters.
- `cp9Agents`: type `Character**`, a dynamic array storing CP9 agents. The number of agents does not exceed 7.
- `cp9Count`: type `int`, the current number of CP9 agents.
- `buildings`: type `Building**`, a dynamic array storing buildings on the battlefield. The number of buildings does not exceed 5.

- `buildingCount`: type `int`, the current number of buildings.
- `turnOrder`: a singly linked list managing the turn order of characters.
- `context`: type `BattleContext`, the current state of the battle.
- `maxTurns`: type `int`, the maximum number of turns allowed in the battle.

2. A constructor that takes the path to a configuration file. This constructor calls the method `loadFromFile` to read data and initialize the battle.

```
1 EniesLobbyBattle(const string& filename);
```

3. A destructor responsible for releasing all dynamically allocated memory for characters, buildings, dynamic arrays, and the linked list.

```
1 ~EniesLobbyBattle();
```

4. Method `loadFromFile` reads data from the file named `filename` and initializes the entire battle.

```
1 void loadFromFile(const string& filename);
```

The input file consists of multiple lines, each describing a group of data. The lines may appear in any order but must follow one of the formats below:

```
1 CONTEXT morale alarmLevel rescueProgress escapeProgress busterCallTimer  
    maxTurns  
2 STRAW_HAT name hp atk def speed energy bounty  
3 CP9 name hp atk def speed energy doriki  
4 BUILDING name hp
```

The `CONTEXT` line initializes the initial values of `BattleContext` and `maxTurns`. The `STRAW_HAT` line creates a Straw Hat character. The `CP9` line creates a CP9 agent. The `BUILDING` line creates a building on the battlefield.

After reading all valid data, the method must construct `turnOrder` by calling `buildTurnOrder`.

5. Method `addStrawHat` adds a Straw Hat character to the dynamic array `strawHats`.

```
1 void addStrawHat(Character* character);
```

6. Method `addCP9Agent` adds a CP9 agent to the dynamic array `cp9Agents`.

```
1 void addCP9Agent(Character* character);
```

7. Method `addBuilding` adds a building to the dynamic array `buildings`.

```
1 void addBuilding(Building* building);
```

8. Method `buildTurnOrder` constructs the turn order list. This list contains only pointers of type `Character*`; buildings are not included in the turn order.

```
1 void buildTurnOrder();
```

The initial order of `turnOrder` is created as follows: all Straw Hat characters in the order they are read from the file are inserted first, followed by all CP9 agents in the order they are read.

Each node in the linked list contains a pointer of type `Character*` and a pointer to the next node.

9. Method `runBattle` executes the entire battle.

```
1 void runBattle();
```

The battle is processed turn by turn until `battleEnded` becomes `true` or the number of turns reaches `maxTurns`. The procedure for each turn is as follows:

- (a) Retrieve the first character in `turnOrder`.
- (b) If the character has been defeated, skip this turn and proceed to the next step.
- (c) If the character is still alive:
 - Determine the target according to the target selection rules.
 - If the character has sufficient energy, perform `specialSkill`.
 - Otherwise, perform `attack`.
 - After the action is completed, call `endTurn`.
- (d) Move the current node to the end of the `turnOrder` list.
- (e) Call `processBuildings` to apply the effects of all buildings.
- (f) Increment `turnCount` by 1.
- (g) Call `checkEndCondition` to evaluate the battle state.

If the number of turns reaches `maxTurns` and the battle has not ended, `battleEnded` is set to `true` and `resultCode` is set to `TIME_OUT`.

10. Method `processTurn` processes the action of a character.

```
1 void processTurn(Character* character);
```

If `character` is not alive, no action is performed. If `character` belongs to the Straw Hat Pirates, the target is selected according to the Straw Hat target selection rules. If `character` belongs to CP9, the target is selected according to the CP9 target selection rules.

If the target is a character, the damage is applied using `receiveDamage`. If the target is a building, the damage is applied using `receiveDamage`; if the building is destroyed as a result, `onDestroyed` is called.

11. Method `processBuildings` applies the effects of all buildings after each turn.

```
1 void processBuildings();
```

For each building in the `buildings` array, call the method `applyEffect`. If a building has been destroyed, its primary effect is no longer applied.

12. Method `checkEndCondition` checks the conditions for ending the battle.

```
1 void checkEndCondition();
```

The ending conditions are checked in the following order:

Order	Condition	Result
1	<code>robinRescued = true</code> and <code>escapeProgress >= 100</code>	<code>STRAW_HAT_WIN</code>
2	<code>busterCallTimer <= 0</code>	<code>BUSTER_CALL</code>
3	All characters in <code>strawHats</code> are defeated	<code>CP9_WIN</code>
4	All characters in <code>cp9Agents</code> are defeated	<code>STRAW_HAT_WIN_BY_DEFEAT_CP9</code>
5	<code>turnCount >= maxTurns</code>	<code>TIME_OUT</code>

13. Method `getResult` returns the result string after the battle ends.

```
1 string getResult() const;
```

The result string has the format:

```
1 resultCode turnCount morale alarmLevel rescueProgress escapeProgress  
  busterCallTimer
```

Example:

```
1 STRAW_HAT_WIN 32 78 25 100 100 4
```

Battle Rules:

1. Target selection rules for the Straw Hat Pirates:

- If `MainGate` is not destroyed, the target is `MainGate`.
- If `MainGate` is destroyed, `alarmLevel >= 50`, and `Courthouse` is not destroyed, the target is `Courthouse`.
- If the above conditions are not satisfied and `busterCallTimer <= 5` and `BusterCallShip` is not destroyed, the target is `BusterCallShip`.
- If none of the above conditions are satisfied and Robin has not been rescued, the target is the first alive CP9 agent in the `cp9Agents` array.
- If Robin has been rescued, the target is `BridgeOfHesitation`. In this case, the character's action is considered as supporting the escape; if this building is destroyed or does not exist, the character attacks the first alive CP9 agent.

- If the current character is Chopper and has enough energy to use **specialSkill**, the target is the Straw Hat member with the lowest **hp**.
2. Target selection rules for CP9: during the turn of a CP9 agent, the target is always the first alive Straw Hat member in the **strawHats** array. CP9 agents do not attack buildings.
 3. Skill usage rules:
 - During a character's turn, if the character has enough energy to use **specialSkill**, it will use **specialSkill**. Otherwise, it will use **attack**.
 - The energy cost of each skill is specified in the corresponding character class. After using a skill, the character's energy must decrease by the corresponding cost. If there is not enough energy, the skill cannot be executed.
 4. Character attack rules:
 - When the target is a character, the damage produced by **attack** or **specialSkill** is applied to the target. If the target's health becomes less than or equal to 0, it is set to 0 and the target is considered defeated.
 - If the attacker belongs to the Straw Hat Pirates and defeats the target, **morale** increases by 5. If the attacker belongs to CP9 and defeats the target, **morale** decreases by 5.

4 Requirements

To complete this major assignment, students must:

1. Read the entire description file.
2. Download the **initial.zip** file and extract it. After extraction, students will receive the files: **main.cpp**, **main.h**, **eniesLobby.h**, **eniesLobby.cpp**, and sample data files. Students must submit only **eniesLobby.h** and **eniesLobby.cpp** and must not modify **main.h** when testing the program.
3. Students should use the following command to compile the program:

```
g++ -o main main.cpp eniesLobby.cpp -I . -std=c++11
```

To run the program, use the command:

```
./main
```

These commands should be used in the command prompt/terminal for compilation and execution. If students use an IDE, they must ensure:

- All necessary files are added to the project/workspace.

- The compilation command includes `eniesLobby.cpp`, `-std=c++11`, and `-I ..`.
 - IDEs typically provide **Build** and **Run** buttons. When pressing **Build**, the IDE executes a corresponding compilation command, often compiling only `main.cpp`. Students must configure the IDE to modify the compilation command to include the necessary files and options.
4. The program will be graded on a Unix-based platform. The grading platform and the student's compiler may differ. The LMS submission system is configured to match the grading environment. Students must test their program in the LMS submission system and resolve any errors there to ensure correct results in the final grading.
 5. Modify `eniesLobby.h` and `eniesLobby.cpp` to complete this assignment while ensuring the following:
 - The only `#include` directive allowed in `eniesLobby.h` is: `#include "main.h"` and in `eniesLobby.cpp`: `#include "eniesLobby.h"`. No other `#include` statements are allowed in these files.
 - Implement all functions as described in the tasks within this assignment.
 - Ensure all classes follow the fundamental principles of Object-Oriented Programming.
 6. Students are encouraged to create additional classes and methods to complete the assignment.
 7. Students must implement all classes and methods listed in this assignment. Failure to do so will result in a score of 0. If a method or class cannot be fully implemented, students must at least provide an empty implementation using curly braces `{}`.

5 Submission

Students must submit only two files: `eniesLobby.h` and `eniesLobby.cpp` before the deadline specified in "Assignment 2 - Submission". Some basic test cases will be used to check whether the submission compiles and runs correctly. Students can submit multiple times, but only the last submission will be graded.

Due to system constraints, a high volume of submissions near the deadline may cause technical issues. Students should submit their work as early as possible. Submissions made in the last hour before the deadline are at the student's own risk. Once the deadline passes, the system will close, and no further submissions will be accepted. Submissions through any other method will not be considered.

6 Additional Rules

- Students must complete this assignment independently and prevent others from copying their work. Any violations will be subject to the university's academic integrity policies.
- All decisions made by the assignment instructor are final.
- Students will not receive test cases after grading. However, the assignment score breakdown will be provided.

7 Academic Integrity and Cheating

The assignment must be completed **individually**. A student will be considered to have cheated if:

- There is an abnormal similarity between submitted code. In such cases, **all submissions involved** will be treated as cheating. Students must protect their source code.
- A student's code is submitted by another student.
- The student does not understand their own submitted code, except for the provided starter code. Students may refer to any resources but must fully understand the meaning of every line they write. If a student does not understand the code they are referencing, they are strongly warned **not to use it**; instead, they should apply concepts learned in class.
- The student accidentally submits another student's work under their own account.
- The student uses source code generated by automated tools without understanding it.

If a student is found guilty of cheating, they will receive a **zero for the entire course** (not just the assignment).

NO EXPLANATIONS WILL BE ACCEPTED, AND NO EXCEPTIONS WILL BE MADE!

After submission, some students may be randomly selected for an interview to verify that their work was completed independently.

8 Conclusion

The battle at Enies Lobby marks one of the most significant moments of the Straw Hat Pirates, as they boldly confront the World Government to rescue Nico Robin. From breaking through the Main Gate, advancing into the Courthouse, to confronting the CP9 agents and racing against



time before the Buster Call is activated, the entire sequence forms a tense and highly dramatic chain of events.

In this assignment, students are able to recreate these events by constructing objects and interaction rules within a program, thereby gaining a clearer understanding of how a narrative can be modeled as a structured system.

WE WISH YOU SUCCESS IN RESCUING ROBIN!